A PROJECT REPORT
ON
PROJECT BASED LEARNING
(ADVANCED JAVA LABORATORY / JAVA PROJECTS, 5TH SEMESTER)

KodBTW

Unified Coding Identity & Campus Coding Intelligence Platform

Submitted in partial fulfillment of the requirement for the award of Degree of
Bachelor of Technology in Computer Science & Engineering

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Sajal Goyal	(24EARCS142)
Shri Krishna Chandra Vaishnav	(24EARCS157)
Yug Mittal	(24EARCS189)
Ravi Shah	(24EARCS130)
Suryadev Singh	(24EARCS170)

Project Code: PBL2627-CSE-C-064

Project Guide
Er. Ram Babu Buri

Session: 2026-27

Department of Computer Science & Engineering
ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

Department of Computer Science & Engineering

DECLARATION

We "Sajal Goyal, Shri Krishna Chandra Vaishnav, Yug Mittal, Ravi Shah and Suryadev Singh", students of **Bachelor of Technology in Computer Science & Engineering, session 2026-27, Arya College of Engineering & Information Technology, Jaipur**, hereby declare that the work presented in this Project entitled "**KodBTW: Unified Coding Identity & Campus Coding Intelligence Platform**" (Project Code PBL2627-CSE-C-064) is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Sajal Goyal	(24EARCS142)
Shri Krishna Chandra Vaishnav	(24EARCS157)
Yug Mittal	(24EARCS189)
Ravi Shah	(24EARCS130)
Suryadev Singh	(24EARCS170)

Date : ___ / __ / _____

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the work embodied in this Project entitled "**KodBTW: Unified Coding Identity & Campus Coding Intelligence Platform**" being submitted by "**Sajal Goyal, Shri Krishna Chandra Vaishnav, Yug Mittal, Ravi Shah and Suryadev Singh**" in partial fulfillment of the requirement for the award of "**Bachelor of Technology in Computer Science & Engineering**" to Rajasthan Technical University, Kota (Raj.) during the academic year 2026-27 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the "**Department of Computer Science & Engineering**", Arya College of Engineering & Information Technology, Jaipur.

Project Guide
Er. Ram Babu Buri
Assistant Professor

Signature: _____

HOD
Dr. Akhil Pandey
Professor

Signature: _____

Department of Computer Science & Engineering

CERTIFICATE OF APPROVAL

The Project Report entitled "**KodBTW: Unified Coding Identity & Campus Coding Intelligence Platform**" submitted by "**Sajal Goyal, Shri Krishna Chandra Vaishnav, Yug Mittal, Ravi Shah and Suryadev Singh**" has been examined by us and is hereby approved for carrying out the project leading to the award of degree "**Bachelor of Technology in Computer Science & Engineering**". By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose.

Project Guide
Er. Ram Babu Buri
Assistant Professor

Signature: _____

Department of Computer Science & Engineering

Acknowledgement

We would like to express our sincere appreciation to our project guide, **Er. Ram Babu Buri**, Department of CSE, for the guidance and feedback provided during the development of "**KodBTW: Unified Coding Identity & Campus Coding Intelligence Platform**". His direction in Java application development and in structuring the project as a Spring Boot and React application helped us carry the work through the project period from 6 July 2026 to 6 October 2026.

We also extend our gratitude to the Department of Computer Science & Engineering at Arya College of Engineering & Information Technology, Jaipur, for providing the academic environment and facilities for this Project Based Learning course. Finally, we thank our teammates and peers for their collaboration and constructive criticism, which helped us complete this project.

Department of Computer Science & Engineering

Abstract

KodBTW is a web application for managing a developer's coding-platform handles and viewing selected profile statistics in one place. It combines account authentication, developer profiles, supported platform synchronisation, dated statistics, dashboards, analytics, history, leaderboard views, contest-calendar information and public developer profiles.

The frontend is a React and TypeScript single-page application. The backend is a Java 21 / Spring Boot 3.3 REST API using Spring Security, stateless JWT authentication, Spring Data JPA, MySQL 8 and Flyway migrations. Provider clients and adapters map available external fields into a normalised, nullable statistics model, and observations are stored as dated, source-labelled snapshots.

The current source enables normal statistics synchronisation for LeetCode, Codeforces and CodeChef. CodeChef uses an unofficial third-party provider with unresolved permission, reliability and rights questions. GeeksforGeeks and HackerRank prototype files exist, but those platforms are disabled by the normal sync gate; the product is therefore not described as supporting live statistics for all five platforms.

This report is an account of an engineering artifact. It makes no claim of learning-outcome benefit, user-study results, production deployment or provider authorisation, because separate evidence for these is not part of the project record. Testing results are reported as attributed historical records.

Keywords: *coding identity, Spring Boot, React, JWT, Flyway, MySQL, data provenance, analytics dashboard, leaderboard*

Department of Computer Science & Engineering

INDEX

Contents **Page No.**

Preliminary Pages
Declaration ii
Certificate iii
Certificate of Approval iv
Acknowledgement v
Abstract vi

Project Report
1 Introduction and Project Context 1
1.1 Background 1
1.2 Project Overview 1
1.3 Project Identity 1
1.4 Project Team 1
1.5 Nature of This Report 2
1.6 Organisation of the Report 2
2 Problem Statement, Motivation, Objectives and Scope 2
2.1 Problem Statement 2
2.2 Motivation 2
2.3 Objectives 3
2.4 Scope 3
2.5 Non-Goals and Exclusions 3
3 Existing Workflow and Literature Context 3
3.1 Existing Workflow 3
3.2 Literature Context 3
3.3 Implications for KodBTW 4
4 Requirements 4
4.1 Functional Requirements 4
4.2 Non-Functional Requirements 4
4.3 Constraints and Assumptions 5
5 Development and Analysis Method 5
5.1 Development Approach 5
5.2 Analysis Approach 5
5.3 Tools and Practices 5
5.4 Project Timeline 5
6 System Architecture 6
6.1 Architectural Pattern 6
6.2 Request Path 6
6.3 Technology Stack 6
6.4 Code Organisation 7
7 Functional Modules and Implementation 7
7.1 Authentication and Identity 7
7.2 Profiles and Public Sharing 7

7.3 Platform Accounts and Synchronisation 7
7.4 Platform Adapters and Integration Status 8
7.5 Dashboard, Analytics and History 9
7.6 Leaderboard 9
7.7 Contest Calendar 9
7.8 Frontend Application 9
8 Data Model and Database Design 9
8.1 Entities 9
8.2 Null Semantics and Source Labels 10
8.3 Flyway Migration Sequence 10
9 API Design and Primary Workflows 11
9.1 Access Model 11
9.2 Registration and Login Workflow 11
9.3 Connect-and-Sync Workflow 11
9.4 Review-and-Share Workflow 12
10 Security, Privacy and Responsible Data Acquisition 12
10.1 Security Controls 12
10.2 Privacy 12
10.3 Ownership of External Handles 12
10.4 Responsible Data Acquisition 12
11 User Experience and Interface Design 12
11.1 Design Goals 12
11.2 Main Views 13
11.3 Interface Evidence 13
12 Testing Strategy and Attributed Historical Results 13
12.1 Strategy 13
12.2 Historical Results 14
12.3 Interpretation 14
13 Build, Configuration and Deployment Capability 14
13.1 Runtime Configuration 14
13.2 Running Locally 15
13.3 Packaging and Deployment Capability 15
14 Discussion, Limitations and Risks 15
14.1 Discussion 15
14.2 Limitations 15
14.3 Risk Register 15
15 Future Work and Conclusion 16
15.1 Future Work 16
15.2 Conclusion 16
References 16
Appendix A: API Inventory 18
Appendix B: Data Invariants 18
Appendix C: Glossary 19
Appendix D: Items Requiring Verification Before Submission 19
Appendix E: Application Snapshots 21

1 Introduction and Project Context

1.1 Background

Programmers who practise on public coding platforms usually spread their activity across several independent services. Each service keeps its own profile page, its own rating scale and its own definition of what counts as a solved problem. A learner or job-seeker who wants a single picture of their coding identity must open each service in turn, copy figures by hand, and mentally reconcile metrics that were never designed to be compared.

KodBTW – Unified Coding Identity & Campus Coding Intelligence Platform is a web application built to reduce that friction. It lets a user keep the handles they use on supported platforms in one account, synchronise the statistics that an enabled source actually provides, store those observations as dated and source-labelled snapshots, and view them through a dashboard, analytics and history views, a leaderboard, a contest calendar and an optional public profile.

1.2 Project Overview

The frontend is a React and TypeScript single-page application. The backend is a Java 21 / Spring Boot 3.3 REST API that uses Spring Security with stateless JWT authentication, Spring Data JPA, MySQL 8 and Flyway migrations. Provider clients and adapters map the fields that each external source exposes into a normalised, nullable statistics model. A field that a provider does not supply stays null; it is never replaced with a fabricated value.

The name of the platform refers to “campus coding intelligence”. This report is deliberately precise about that phrase: campus-wide monitoring and mentor dashboards are **not** part of the verified implementation. What is implemented is an individual-centred aggregation tool together with college-filtered leaderboard reads.

1.3 Project Identity

Table 1.1: Project identity

Item	Details
Title	KodBTW – Unified Coding Identity & Campus Coding Intelligence Platform
Project code	PBL2627-CSE-C-064
Institution	Arya College of Engineering & I.T., Jaipur, Rajasthan
Course / programme	Advanced Java Laboratory / Project Based Learning (PBL), Java Projects, 5th Semester
Academic year	2026–27
Project period	6 July 2026 to 6 October 2026
Project guide	Er. Ram Babu Buri, Department of CSE
Project type	Website application (frontend, REST API, relational persistence)
Source repository	https://github.com/sajalgoyal007/KodBTW

1.4 Project Team

The project files list the following five team members. Enrolment identifiers are reproduced from the team's earlier project report and must be checked against college records before submission. Individual module ownership is not confirmed in the current contribution record; this report therefore lists the roster only and does not assign work to individuals.

Table 1.2: Team roster

S. No.	Team member	Enrolment ID (to be verified)	Contribution record
1	Sajal Goyal	24E1ARCSM40P142	Team Lead, API and authentication
2	Shri Krishna Chandra Vaishnav	24E1ARCSM30P157	Frontend application and user experience
3	Yug Mittal	24E1ARCSM40P189	Platform integrations and statistics synchronization
4	Ravi Shah	24E1ARCSM30P130	Database, persistence and analytics
5	Suryadev Singh	24E1ARCSM40P170	Quality, contest calendar and project testing

1.5 Nature of This Report

KodBTW is presented here as an engineering artifact. The report describes what the source code and project documents establish. It does not claim a learning-outcome benefit, the result of a user study, a production deployment, uptime, or authorisation from any external provider, because no evidence for those claims is part of the project record. Where the record is incomplete or inconsistent, the report says so.

1.6 Organisation of the Report

Chapter 2 states the problem, objectives and scope. Chapter 3 reviews the existing workflow and the literature context. Chapters 4 and 5 give the requirements and the development method. Chapters 6 to 9 describe the architecture, modules, data model and API. Chapter 10 covers security, privacy and responsible data acquisition, and Chapter 11 the user interface. Chapters 12 and 13 report testing and build/configuration. Chapters 14 and 15 discuss limitations, risks, future work and the conclusion. The appendices contain the API inventory, data invariants, a glossary and a pre-submission verification list.

2 Problem Statement, Motivation, Objectives and Scope

2.1 Problem Statement

Users who practise across multiple coding platforms must switch between separate services to review their coding identity and progress. Each platform exposes different fields and defines its metrics differently. A unified view is only useful if it also communicates **where** a figure came from, **when** it was observed, **which fields are unavailable**, and **how far** figures from different sources can be compared.

2.2 Motivation

- **Fragmentation:** a coding profile is scattered over several services, none of which shows the whole picture.
- **Loss of history:** a platform page typically shows the current state, so a user who wants to see progress over time has to record it manually.
- **Misleading aggregation:** naïvely summing or averaging platform metrics can hide missing data and mix incompatible definitions; a trustworthy tool must keep provenance and missingness visible.
- **Shareable identity:** a developer may want a single public page that presents selected profile information in a controlled way.

2.3 Objectives

1. Provide registration, login, email verification and authenticated user identity.
2. Let users maintain a profile and a unique public username.
3. Let users manage their own coding-platform handles.
4. Synchronise only platforms with an enabled source and persist the supported fields.
5. Store dated, source-labelled observations for history and analytics.
6. Provide dashboard, analytics, activity, history, leaderboard and contest-calendar views.
7. Provide an explicitly shaped public-profile response.
8. Represent missing and unavailable data truthfully rather than substituting fabricated values.

2.4 Scope

Implemented or represented in source: authentication, user and profile APIs, the platform-account lifecycle, adapter-driven synchronisation, snapshots, analytics and history services, leaderboard cache and read APIs, the contest calendar, public profile views, and responsive frontend routes.

2.5 Non-Goals and Exclusions

The following are **not established** by the project and are outside the scope of this report: code submission or execution; universal cross-platform metric normalisation; proof of ownership of every external handle; campus or mentor monitoring; predictive recommendations; AI coaching; production uptime; a completed user study; and authorisation for every automated provider request.

3 Existing Workflow and Literature Context

3.1 Existing Workflow

Without an aggregation tool, a user follows a manual workflow: open each platform, find the profile page, read the figures that the platform chooses to show, and record them elsewhere if history is wanted. The workflow has three weaknesses. It does not scale as the number of platforms grows; it produces no consistent timeline; and it gives no indication of which figures are comparable. KodBTW replaces the manual steps with account-scoped synchronisation and stored snapshots, while keeping the provider and the time of every observation attached to the data.

3.2 Literature Context

The project's literature review covers 30 works on learner-facing dashboards, learning analytics, programming-process data, online judges and automated feedback. Eleven of them are listed in the References chapter. The works fall into four groups.

- **Dashboard research** (Verbert et al., 2013; Schwendimann et al., 2017; Bodily and Verbert, 2017; Jivet et al., 2018; Valle et al., 2021) surveys how learner-facing dashboards are designed and evaluated, and stresses that a dashboard should be evaluated rather than assumed to be helpful.
- **Programming-specific analytics** (Ihantola et al., 2016; Altadmri and Brown, 2015; Hellings and Haelermans, 2022) studies data from programming courses. Many such studies use rich traces such as compilations, submissions or course interactions.
- **Online judges** (Kurnia et al., 2001; Wang et al., 2023) examine judge systems and how problem difficulty and order relate to programming-education outcomes.
- **Automated feedback** (Keuning et al., 2018) reviews feedback generation for programming exercises.

3.3 Implications for KodBTW

The narrow conclusion that the literature supports is that careful dashboard design and evaluation matter. It does **not** show that KodBTW improves learning. The prior studies often rely on richer code and submission traces than the public profile statistics available to KodBTW, and a dashboard is not by itself an educational intervention. For that reason the report treats KodBTW as an engineering contribution and limits its claims accordingly. The design choices that follow from the literature are modest: show provenance, keep missing data visible, avoid composite scores that imply false comparability, and leave any evaluation of educational benefit as future work.

4 Requirements

4.1 Functional Requirements

Table 4.1: Functional requirements

ID	Requirement	Module
FR-1	Register, log in, verify email and retrieve the authenticated identity; Google sign-in and account linking flows are represented.	Authentication
FR-2	Read and update an owned profile; set a unique public username; opt in to profile search indexing.	Profiles
FR-3	Add, list, inspect, update and delete own platform accounts (handle per platform).	Platform accounts
FR-4	Synchronise an owned account when its source is enabled; enforce cooldown and concurrency rules; classify failures.	Sync
FR-5	Persist normalised, dated, source-labelled snapshots; preserve earlier successful snapshots on failure.	Snapshots
FR-6	Show dashboard statistics, analytics, history, insights and activity for the authenticated owner.	Dashboard
FR-7	Provide paginated leaderboard reads, college-filtered reads and the current user's rank; admin-only cache rebuild.	Leaderboard

ID	Requirement	Module
FR-8	Show a contest calendar from the default Codeforces provider; allow an explicit local fixture mode.	Contests
FR-9	Expose a sanitised public profile by username and a public sitemap.	Public profile

4.2 Non-Functional Requirements

- **Security:** stateless JWT; most routes authenticated; owner-scoped operations; secrets supplied at runtime.
- **Data integrity:** unique constraints for account and snapshot identity; schema controlled by versioned migrations.
- **Truthfulness:** null means "no supported value recorded", never zero; every snapshot carries a source label.
- **Maintainability:** layered code (controllers, services, adapters, repositories) and automated tests on both tiers.
- **Usability:** responsive layouts, light and dark theme, keyboard and accessibility markup with automated contrast and markup checks.

4.3 Constraints and Assumptions

External platforms control their own data contracts, terms and availability. Some of the data sources used are undocumented or unofficial, so provider behaviour can change without notice. The project period (6 July to 6 October 2026) and the availability of a five-member student team bound the amount of functionality that could be built and verified.

5 Development and Analysis Method

5.1 Development Approach

The team developed the system as a conventional layered web application with a separate frontend and backend in one repository. Work was tracked through Git; the repository history contains 64 commits on the `main` branch at the time of writing, authored under at least two team members' names. Commit authorship and counts do not establish complete task ownership, which is why individual contributions are left to team verification.

5.2 Analysis Approach

Requirements were derived by comparing the manual workflow (Chapter 3) with the fields that each provider can supply. This analysis produced three design principles that recur throughout the code base: (1) provenance is stored with every observation; (2) unavailable data is represented as null or as an "unavailable/pending" status, never as a substituted value; and (3) any operation on an account is checked against the authenticated owner.

5.3 Tools and Practices

Table 5.1: Tools and practices

Area	Tool / practice
Version control	Git and GitHub (public repository, `main` branch)

Area	Tool / practice
Backend build	Maven wrapper; Java 21
Frontend build	npm; Vite 5.4
Schema evolution	Flyway migrations; Hibernate `ddl-auto: validate`
Backend tests	Spring Boot Test, JUnit, H2, Spring Security Test, Testcontainers
Frontend tests	Node.js built-in test runner
Continuous integration	GitHub Actions workflow for the backend (test and package; requires Docker for Testcontainers)

5.4 Project Timeline

The project ran from 6 July 2026 to 6 October 2026. The repository's weekly-report index exists, but individual weekly reports and daily progress logs were not found in the repository. This report therefore does not reconstruct a week-by-week plan; doing so would require records that are not available. The migration history (Chapter 8) is the most reliable evidence of how the data model evolved during the period.

6 System Architecture

6.1 Architectural Pattern

KodBTW is a layered, stateless client–server system. The React single-page application talks only to the Spring REST API. The API contains security, controllers, services, platform adapters and repositories, and talks to one MySQL database and to external provider endpoints. There is a single deployable backend; no message queue or additional microservice is part of the verified architecture.

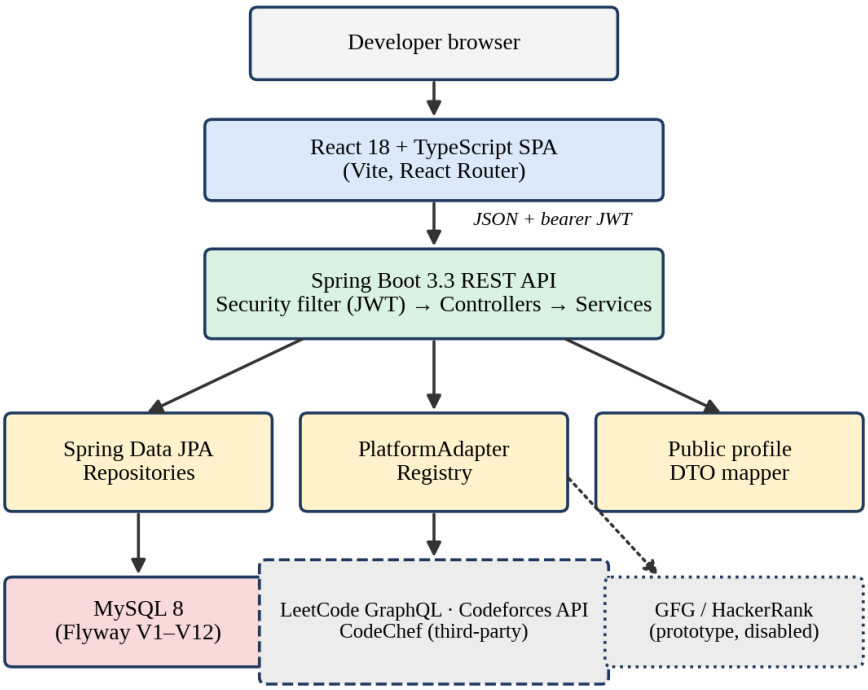


Figure 6.1: Layered architecture of KodBTW

6.2 Request Path

1. The React frontend sends JSON requests to the Spring REST API.
2. Protected calls carry a bearer JWT in the `Authorization` header.
3. Spring Security's JWT filter validates the token and resolves the user.
4. Controllers delegate to services, which perform ownership checks and business logic.
5. Services access JPA repositories and MySQL, or call a platform adapter/client.
6. Successful supported responses are normalised and persisted as source-labelled snapshots.
7. A response DTO returns the result to the frontend.

6.3 Technology Stack

Table 6.1: Technology stack

Layer	Technologies	Responsibility
Frontend	React 18, TypeScript 5.6, React Router 6, Vite 5.4	Single-page UI, routing, charts and views
API	Java 21, Spring Boot 3.3.5, Spring Web, validation	REST endpoints and application services
Security	Spring Security, JWT (JJWT), password encoder (BCrypt)	Stateless authentication and password hashing
Persistence	Spring Data JPA, Hibernate, MySQL Connector, MySQL 8	Entities, repositories, storage
Schema	Flyway; Hibernate validation only	Versioned schema evolution

Layer	Technologies	Responsibility
Backend testing	Spring Boot Test, JUnit, H2, Spring Security Test, Testcontainers	Unit, slice and integration tests
Frontend testing	Node.js built-in test runner	Utility, presentation, markup and request tests

6.4 Code Organisation

Backend code lives under `backend/src/main/java/com/kodbtw/` and is organised into `controller`, `service`, `security`, `config`, `adapter` (including `adapter/contest`), `entity`, `dto` (including `dto/analytics`, `dto/history` and `dto/insights`) and repository packages. Migrations are in `backend/src/main/resources/db/migration/`. The frontend source lives under `frontend/src/` with `App.tsx` (routes), `pages/`, `components/`, `services/` and tests.

7 Functional Modules and Implementation

7.1 Authentication and Identity

The API provides registration, login, email verification and resend-verification routes, Google sign-in and challenge flows, Google account linking, and an authenticated identity lookup (`GET /api/auth/me`). Registration hashes passwords with BCrypt; login returns a signed JWT. Authentication is stateless: form login and HTTP Basic are disabled, and a JWT filter validates the bearer token on every protected request and resolves the user from the database. Google provider identities are kept in a separate table (`UserAuthIdentity`) rather than on the core user, which is why the user's password may be null for federated accounts. Email-verification tokens are stored hashed with an expiry and a used timestamp.

7.2 Profiles and Public Sharing

Authenticated users can read and update their own profile. A unique username supports public lookup. Public data is composed through a dedicated response DTO (`GET /api/public/profiles/{username}`), not by returning the raw user entity, so internal fields such as email and password data cannot leak through the public route. Inclusion in profile search indexing is opt-in. A public sitemap is served under `/api/public`, and the frontend renders public profiles at `/u/:username`.

7.3 Platform Accounts and Synchronisation

An authenticated user can add, list, inspect, update, delete and synchronise their own platform-account records. The service checks ownership, whether the source is supported, concurrency gates and cooldown rules (default cooldown `PT15M`). Successful synchronisation persists normalised data with timestamps. Failures are classified and are meant to preserve earlier successful observations rather than replace them with mock data. Unsupported sources return an unavailable/pending status; they are not converted into verified data.

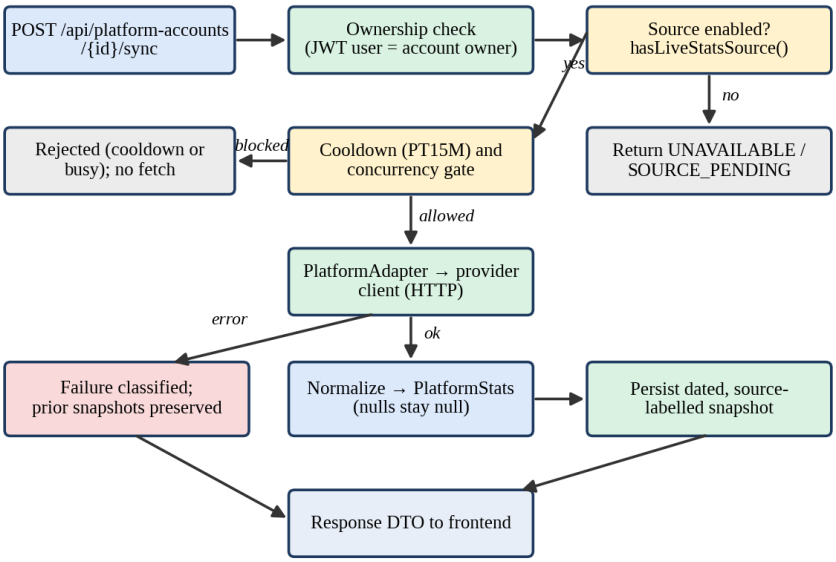


Figure 7.1: Platform synchronisation workflow (conceptual; the service source is authoritative for the exact order of checks)

7.4 Platform Adapters and Integration Status

Synchronisation resolves a `PlatformAdapter` through a `PlatformAdapterRegistry`, calls the provider client, and maps the supported fields into a `PlatformStats` object. `Platform.hasLiveStatsSource()` acts as the gate that decides whether a platform takes part in normal synchronisation.

Table 7.1: Platform integration status

Platform	Normal sync	Source label / status	Caveat
LeetCode	Enabled	`LEETCODE_REAL`	Uses an undocumented public GraphQL query; terms and stability need review. Missing response fields remain null.
Codeforces	Enabled	`CODEFORCES_REAL`	Uses public `user.info`; only mapped fields are populated. Current and maximum rating are separate; solve counts and history may remain null.
CodeChef	Enabled	`CODECHEF_THIRD_PARTY`	Unofficial third-party provider. Permission, upstream rights, reliability and authorisation are unresolved.
GeeksforGeeks	Disabled	Unavailable / pending	Prototype client and adapter files exist but do not pass the live-source gate.
HackerRank	Disabled	Unavailable / pending	Prototype client and adapter files exist but do not pass the live-source gate.

A public web page that returns successfully does not establish authorisation for automated extraction. Nothing in this report implies production clearance for CodeChef or for any other source, and the project record does not show that permission requests were sent or approved. The product must not be described as fully supporting live statistics for all five platforms.

7.5 Dashboard, Analytics and History

Backend endpoint families under `/api/dashboard` include `stats`, `analytics`, `history`, `insights`, `activity` and `contests`. The frontend components include metric cards, platform share, difficulty breakdown, historical charts, platform comparisons, activity and contest intelligence, streak comparisons and insight sections. All of them are owner-scoped. Because the data comes from dated snapshots, history views show change over time without re-querying providers, and values that a source never supplied are displayed as unavailable rather than zero.

7.6 Leaderboard

The leaderboard supports full, college-filtered and current-user-rank reads. A denormalised `LeaderboardUserCache` supports paginated and sortable reads and real-source filtering. `POST /api/leaderboard/sync`, which rebuilds the cache, requires the `ADMIN` role. Because ratings from different platforms are not equivalent, leaderboard values should be read together with their source labels.

7.7 Contest Calendar

`GET /api/contests` is authenticated. The default provider is Codeforces, and results are cached (default `PT10M`). A local fixture provider can be selected explicitly for deterministic development and UI testing; its records are illustrative and are not live contests.

7.8 Frontend Application

The frontend is a React 18 / TypeScript single-page application using React Router 6 and built with Vite 5.4. It provides responsive layouts, a light/dark theme, and keyboard and accessibility markup. During development Vite serves on port 5173 and proxies `/api` to the backend on port 8080. For hosting, a Vercel single-page rewrite configuration is included.

8 Data Model and Database Design

8.1 Entities

Table 8.1: Main entities

Entity	Purpose and key constraints
`User`	Root account; unique email. Password may be null for federated identity.
`Profile`	At most one per user; unique public username; optional profile fields; indexing opt-in.
`PlatformAccount`	Handle and synchronisation lifecycle; unique `(user_id, platform)`.
`PlatformStatSnapshot`	Dated statistics and source; unique user/platform/date combination.
`LeaderboardUserCache`	Optional one-to-one derived leaderboard projection.
`UserAuthIdentity`	Provider-subject mapping with uniqueness constraints.
`GoogleAuthChallenge`	Nonce hash, operation, optional user binding, expiry.
`EmailVerificationToken`	Hashed token, expiration, used timestamp.
`schema_initialization_check`	Migration utility table; not a domain entity.

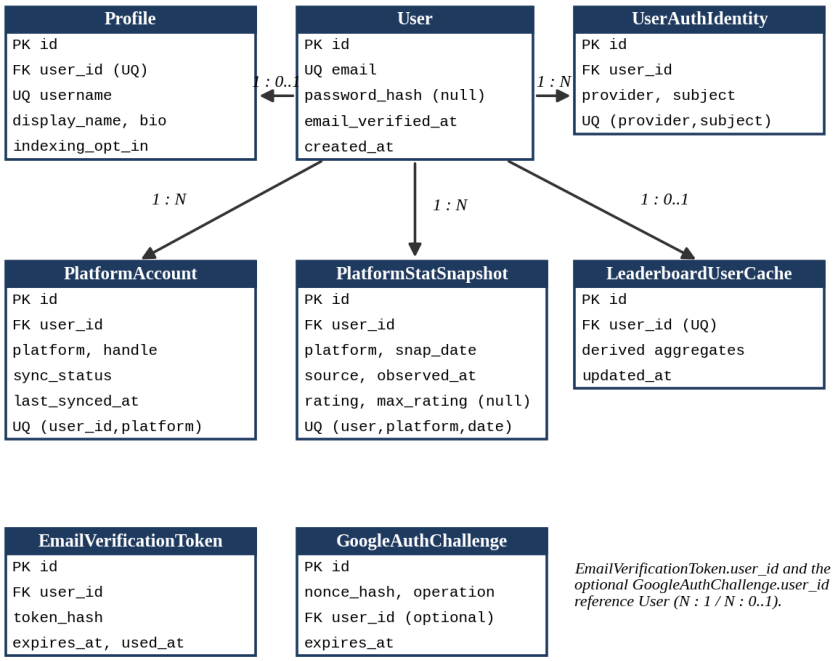


Figure 8.1: Entity-relationship overview. Attribute names are illustrative; the Flyway migrations are the authoritative DDL.

8.2 Null Semantics and Source Labels

Null means that no supported value was recorded; it does not mean zero. A rating of zero and an unknown rating are different facts, and the schema and services keep them apart. Source labels distinguish real provider data (`LEETCODE_REAL`, `CODEFORCES_REAL`), third-party data (`CODECHEF_THIRD_PARTY`) and mock or test data. A user's maximum rating and their rank are stored as separate concepts.

8.3 Flyway Migration Sequence

The repository's current migration directory contains files through **V12**. Flyway validates the migrations, and Hibernate uses `ddl-auto: validate`, so Hibernate checks the mapped schema but never creates or alters it.

Table 8.2: Migration sequence

Migration	Content
V1–V4	Initialisation marker; users; profiles and platform accounts; snapshots and leaderboard cache
V5–V7	Unique username; sync lifecycle and snapshot observation fields; maximum rating separated from rank
V8	Opt-in profile indexing
V9	Federated authentication identity; nullable password
V10–V11	Google challenge data; operation kind and optional user binding
V12	Email-verification columns and token table; indexes; migration policy for existing federated and password users

The older database documentation in the repository (and the README) describes only V1–V7 and should be treated as outdated relative to the current migration directory.

9 API Design and Primary Workflows

9.1 Access Model

All backend routes use the `/api` prefix. Most routes require a valid bearer token; the explicit exceptions are the health endpoint, selected authentication endpoints and the public APIs. `GET /api/health` is a public liveness endpoint and is **not** a database-readiness check.

Table 9.1: API overview

Route	Access and purpose
`POST /api/auth/register`, `POST /api/auth/login`	Public registration and login
`GET /api/auth/me`	Authenticated current user
Email verification, resend, Google sign-in / link routes	Selected public or authenticated authentication flows
`GET /api/health`	Public liveness
`GET/PUT /api/profile`	Authenticated owner profile
`GET /api/public/profiles/{username}`	Public sanitised profile
`/api/platform-accounts/**`	Authenticated, owner-scoped account and sync operations
`/api/dashboard/**`	Authenticated stats, analytics, history, insights, activity
`GET /api/contests`	Authenticated contest calendar
`/api/leaderboard/**`	Authenticated reads; `POST /api/leaderboard/sync` requires `ADMIN`

Controller annotations in `backend/src/main/java/com/kodbtw/controller/` are the source of truth for exact routes and parameters.

9.2 Registration and Login Workflow

The frontend posts credentials to the registration or login route. The backend validates input, hashes the password with BCrypt on registration, and after a successful login returns a signed JWT (default lifetime 24 hours). Subsequent protected requests send the token; the JWT filter validates it and resolves the user. Email verification adds a hashed, expiring token that is marked as used on success.

9.3 Connect-and-Sync Workflow

An authenticated user adds a platform handle. Every later operation on that account checks that the account belongs to the caller. When the source is enabled, a sync request passes the cooldown and concurrency gates, calls the adapter, normalises the supported fields, and writes a dated, source-labelled snapshot together with status and timestamps. Failure paths (cooldown, serialisation conflict, unavailable source and upstream failure) are handled by the sync service, and a failed sync preserves previously stored successful data.

9.4 Review-and-Share Workflow

Dashboard endpoints return owner-scoped metrics and persisted history. A visitor who knows a user's public username can request the sanitised public-profile DTO. Connecting an account does not by itself prove that the user owns the external handle (see Chapter 10).

10 Security, Privacy and Responsible Data Acquisition

10.1 Security Controls

- Stateless JWT authentication; form login and HTTP Basic are disabled.
- Most API routes require authentication; health, selected authentication endpoints and public APIs are explicit exceptions.
- Account operations are scoped to the authenticated owner through service-layer ownership checks.
- Leaderboard rebuild is restricted to the `ADMIN` role.
- Database credentials and the JWT signing secret are external runtime settings and must not be committed. The secret should be at least 32 random bytes.
- CORS origins are configured explicitly and are required in the production profile.

10.2 Privacy

Public profiles use a dedicated DTO and opt-in indexing, which limits what is exposed. Email addresses and tokens must not appear in public responses or in screenshots used as report evidence. Provider terms, data rights, retention and consent require review before production use; the repository does not contain a privacy policy or a consent flow, and this report makes no compliance claim.

10.3 Ownership of External Handles

Account connection is not equivalent to verified ownership of an external handle unless a separate verification flow proves it. The verified implementation does not include such a flow for every provider, so a displayed handle should be read as "handle supplied by the user", not "handle proven to belong to the user".

10.4 Responsible Data Acquisition

The project's platform-acquisition record documents the source for each platform. Three principles apply: use only sources enabled by the live-source gate; label third-party data as such; and treat undocumented or unofficial endpoints as unstable and unresolved with respect to permission. LeetCode's query is undocumented; CodeChef relies on an unofficial provider; GeeksforGeeks and HackerRank are disabled precisely because their sources are unresolved. Before any production use the team should obtain written permission or switch to officially documented interfaces where they exist.

11 User Experience and Interface Design

11.1 Design Goals

The design intent is to make provenance and missing data visible instead of hiding them: values are tied to their platform of origin, unavailable fields are represented as unavailable, and platform

comparisons are not meant to present different rating systems as one scale. These intentions follow the data model; the screenshots required by Table 11.2 would confirm them at runtime. The layout is responsive, supports a light and dark theme, and includes keyboard and accessibility markup that is covered by automated contrast and markup tests. The repository audit is not a full device or browser audit.

11.2 Main Views

Table 11.1 summarises the views implied by the frontend components and backend endpoint families named in the project record.

Table 11.1: Main frontend views

View	Content
Dashboard	Metric cards, platform share, difficulty breakdown
Analytics and history	Historical charts, platform comparisons, streak comparisons
Insights and activity	Insight sections, activity and contest intelligence
Platform accounts	Add, edit, delete and synchronise handles; sync status
Leaderboard	Paginated, sortable, college-filtered ranking with source filter
Contest calendar	Upcoming contests from the configured provider
Public profile (`/u/:username`)	Sanitised public view of a developer's profile

11.3 Interface Evidence

The application snapshots in Appendix E document the deployed landing page and major application views. Displayed statistics and contest listings are point-in-time values and may change when their source data refreshes. Login and public-profile screenshots were not supplied and remain pending.

Table 11.2: Screenshot evidence register (to be completed by the team)

Fig.	Route / scenario	Status
A	Login and registration	Pending capture
B	Dashboard with synchronised data	Captured; see Appendix E, Fig. E.2
C	Analytics and history charts	Captured; see Appendix E, Figs. E.3-E.5
D	Leaderboard	Captured; see Appendix E, Fig. E.7
E	Contest calendar	Captured; see Appendix E, Fig. E.6
F	Public profile	Pending capture
G	Project landing page	Captured; see Appendix E, Fig. E.1
H	Coding profiles	Captured; see Appendix E, Fig. E.8

12 Testing Strategy and Attributed Historical Results

12.1 Strategy

The backend has Java unit, adapter/client, controller, security and Flyway migration tests using Spring Boot Test, JUnit, H2, Spring Security Test and Testcontainers. The frontend has Node.js built-in-runner tests for utilities, presentation, markup and request behaviour. The GitHub Actions workflow runs the backend `test package` job and uploads Surefire reports; Docker is required for the Testcontainers step. No frontend CI job and no end-to-end browser suite are configured.

12.2 Historical Results

No services were started and no tests or builds were run for this report. The figures below are historical records, each reported with its provenance. They are not combined into one current result.

Table 12.1: Historical validation records

Record	Backend	Frontend	Migrations
A – supplied previously by the project owner	309 tests; 0 failures; 0 errors; 2 skipped	110 passed; production build passed	Docker-backed MySQL: V1–V11 passed
B – repository validation document, run of 12 October 2026	Full suite: 262 enumerated; 0 assertion failures; 1 Docker/Testcontainers error; 2 skipped. Non-Docker selection: 261 tests; 0 failures; 0 errors; 2 skipped	70 passed; build passed	Not stated here

12.3 Interpretation

The two records differ in test counts and cannot be reconciled without test logs, timestamps and commit identifiers. Record A's migration statement covers V1–V11 and therefore does **not** establish that V12 applies successfully. Automated results also do not establish production performance, deployment health, external-source authorisation, external data accuracy or manual browser coverage. Before final submission the team should re-run the full suite on a clean checkout with Docker available and record the commit identifier, date and logs.

13 Build, Configuration and Deployment Capability

13.1 Runtime Configuration

Table 13.1: Configuration variables

Variable	Purpose	Requirement / default
`DB_URL`	MySQL JDBC URL	Required
`DB_USERNAME`, `DB_PASSWORD`	Database credentials	Required; least-privilege account
`JWT_SECRET`	JWT HMAC signing key	Required; at least 32 random bytes
`JWT_EXPIRATION_MS`	JWT lifetime	Default 86,400,000 ms (24 h)
`PORT`	Backend HTTP port	Default 8080
`CORS_ALLOWED_ORIGINS`	Allowed browser origins	Explicit in production

Variable	Purpose	Requirement / default
`VITE_API_BASE_URL`	Public API origin compiled into frontend	Public build-time value; no secrets
`CONTEST_CALENDAR_PROVIDER`	Contest source	`codeforces` default; `fixture` local only
`CONTEST_CALENDAR_CACHE_TTL`	Contest cache age	`PT10M` default
`PLATFORM_SYNC_COOLDOWN`	Per-account sync cooldown	`PT15M` default

13.2 Running Locally

Prerequisites are Java 21, MySQL 8, Node.js 18+ with npm, and Docker for Testcontainers tests. After creating an empty `kodbtw` database, the developer sets the backend variables in the process environment (the `.env.example` file is only a reference and is not loaded automatically), then runs the Maven wrapper from `backend/` (`test`, `package`, `spring-boot:run`) and `npm ci` followed by `npm run dev` from `frontend/`.

13.3 Packaging and Deployment Capability

The backend Dockerfile uses Java 21 and a non-root runtime user. The frontend includes a Vercel single-page rewrite configuration, and production settings are described in `application-prod.yml`. These files show **deployment capability**; they are not proof of a deployed hosting account or a production system. The repository's About field links to `kodbtw.vercel.app`, but this report did not verify that the site is live, and no uptime claim is made.

14 Discussion, Limitations and Risks

14.1 Discussion

The strongest aspects of the project are its explicit treatment of provenance and missingness, owner-scoped service logic, versioned schema evolution and the separation of public and private response shapes. Its weakest aspects are the dependence on unstable or unofficial data sources and the lack of runtime evidence in the form of screenshots and a current, reconciled test record.

14.2 Limitations

- Upstream schemas, terms, rate limits and availability can change at any time.
- Available metrics differ by platform; ratings from different platforms are not equivalent.
- Only three of five platforms take part in normal synchronisation.
- The frontend uses client-side routing; unknown-route HTTP status behaviour and server-rendered metadata are not implemented.
- No user study, performance measurement or production deployment evidence exists.

14.3 Risk Register

Table 14.1: Risks and mitigations

Risk	Impact	Mitigation
Provider terms or permission unresolved (LeetCode, CodeChef)	Legal or availability problems if used in production	Seek permission or official interfaces; label sources; keep disabled sources off
Provider API change	Broken sync or wrong mapping	Null-safe mapping; failure classification; keep prior snapshots
Unverified handle ownership	Users may claim other people's handles	Add an ownership-verification flow before relying on identity
Documentation drift (V1–V7 documents vs V1–V12 migrations)	Misleading reviewers	Update database documentation from current migrations
Inconsistent test records	Weak validation evidence	Re-run on a clean checkout; archive logs and commit ID
No genuine screenshots	Weak interface evidence	Capture redacted screenshots from a running instance
Health endpoint is liveness only	False sense of readiness	Add a database-aware readiness check

15 Future Work and Conclusion

15.1 Future Work

The following are possible directions, subject to team approval and to platform authorisation. They are **not** claims of completed work.

- Move to official or explicitly approved data contracts for each platform, and enable GeeksforGeeks and HackerRank only after their sources are cleared.
- Add an external-handle ownership verification flow.
- Add frontend CI, richer accessible browser/end-to-end tests and a readiness endpoint.
- Server-render public-profile metadata for sharing and search.
- Evaluate whether the dashboards help users, through a properly designed study, before making any learning claims.

15.2 Conclusion

KodBTW consolidates selected coding-platform records into one authenticated, owner-scoped application and keeps source, time and missingness attached to every observation. The implemented system combines a React/TypeScript client with a Spring Boot REST API, JWT security, MySQL persistence under Flyway migrations V1–V12, adapter-driven synchronisation for LeetCode, Codeforces and CodeChef, dated snapshots, dashboards, a leaderboard, a contest calendar and sanitised public profiles. Its limits are equally part of the result: some sources are unofficial or undocumented, two platforms remain disabled, the test record is historical and inconsistent, and no learning benefit or production deployment is claimed. Within those limits the project demonstrates a careful engineering approach to aggregating heterogeneous public data.

References

The following are selected from the project's bibliography of 30 works.

[1] K. Verbert et al., "Learning analytics dashboard applications," American Behavioral Scientist, vol. 57, no. 10, pp. 1500–1509, 2013. doi:10.1177/0002764213479363.

[2] B. Schwendimann et al., "Perceiving Learning at a Glance: A Systematic Literature Review of Learning Dashboard Research," IEEE Transactions on Learning Technologies, vol. 10, no. 1, pp. 30–41, 2017. doi:10.1109/TLT.2016.2599522.

[3] R. Bodily and K. Verbert, "Review of Research on Student-Facing Learning Analytics Dashboards and Educational Recommender Systems," IEEE Transactions on Learning Technologies, vol. 10, no. 4, pp. 405–418, 2017. doi:10.1109/TLT.2017.2740172.

[4] I. Jivet et al., "License to evaluate: preparing learning analytics dashboards for educational practice," in Proc. LAK 2018, pp. 31–40. doi:10.1145/3170358.3170421.

[5] N. Valle et al., "Staying on target: A systematic literature review on learner-facing learning analytics dashboards," British Journal of Educational Technology, vol. 52, pp. 1724–1748, 2021. doi:10.1111/bjet.13089.

[6] J. Hellings and C. Haelermans, "The effect of providing learning analytics on student behaviour and performance in programming: a randomised controlled experiment," Higher Education, vol. 83, pp. 1–18, 2022. doi:10.1007/s10734-020-00560-z.

[7] P. Ihantola et al., "Educational data mining and learning analytics in programming: Literature review and case studies," ITiCSE Working Group Reports, pp. 41–63, 2016. doi:10.1145/2858796.2858798.

[8] A. Altadmri and N. C. C. Brown, "37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data," in Proc. SIGCSE 2015, pp. 522–527. doi:10.1145/2676723.2677258.

[9] J. Wang et al., "How problem difficulty and order influence programming education outcomes in online judge systems," Heliyon, vol. 9, no. 11, e20947, 2023. doi:10.1016/j.heliyon.2023.e20947.

[10] H. Keuning, J. Jeuring and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," ACM Transactions on Computing Education, vol. 19, no. 1, 2018. doi:10.1145/3231711.

[11] A. Kurnia, A. Lim and B. Cheang, "Online judge," Computers & Education, vol. 36, no. 4, pp. 299–315, 2001. doi:10.1016/S0360-1315(01)00018-5.

Project Evidence

Repository: https://github.com/sajalgoyal007/KodBTW (main branch). Key documents consulted: `README.md`; `docs/ARCHITECTURE.md`; `docs/platform-data-acquisition.md`; `docs/contest-calendar-fixture-mode.md`; `documentation/PROJECT_AUDIT.md`; `documentation/TEAM_CONTRIBUTIONS.md`; `documentation/TESTING_AND_VALIDATION.md`; the architecture, database, system-design and UML diagram notes under `documentation/diagrams/`; the Flyway scripts in `backend/src/main/resources/db/migration/`; and the earlier project deliverables (SRS, literature review, research paper and a 14-page final report). Where the older database documentation (V1–V7) differs from the migration directory (V1–V12), the migrations are treated as authoritative.

Appendix A: API Inventory

Route families as described in the project record. Controller sources are authoritative for exact paths, methods and parameters.

Table A.1: Route families

Family	Access	Notes
`/api/auth/*`	Public (register, login, verification, Google flows) and authenticated (`me`, linking)	JWT returned on login; email verification token hashed and expiring
`/api/health`	Public	Liveness only, not database readiness
`/api/profile`	Authenticated owner	Read and update
`/api/public/profiles/{username}`	Public	Dedicated sanitised DTO
`/api/public` (sitemap)	Public	Opt-in indexed profiles
`/api/platform-accounts/**`	Authenticated, owner-scoped	Add, list, inspect, update, delete, sync
`/api/dashboard/stats`	Authenticated	Summary metrics
`/api/dashboard/analytics`	Authenticated	Difficulty, platform, comparison data
`/api/dashboard/history`	Authenticated	Dated snapshot history
`/api/dashboard/insights`	Authenticated	Insight sections
`/api/dashboard/activity`	Authenticated	Activity and streaks
`/api/dashboard/contests`, `/api/contests`	Authenticated	Contest calendar; Codeforces default; fixture optional
`/api/leaderboard/**`	Authenticated	Leaderboard, college filter, own rank
`POST /api/leaderboard/sync`	`ADMIN` only	Rebuilds leaderboard cache

Appendix B: Data Invariants

- A user has a unique email; a profile has a unique username and belongs to exactly one user.
- A user has at most one platform account per platform: unique `(user_id, platform)`.
- A user has at most one snapshot per platform per date: unique user/platform/date.
- Null in a snapshot field means “no supported value recorded”, never zero.
- Every snapshot carries a source label that distinguishes real, third-party and mock/test data.
- Maximum rating and rank are separate fields.
- A failed synchronisation never overwrites an earlier successful snapshot with mock data.
- The leaderboard cache is derived data with at most one row per user and can be rebuilt.
- A federated user may have no password; provider identity is unique per provider and subject.
- Email-verification and Google-challenge tokens are stored hashed (token or nonce hash), with expiry.

Appendix C: Glossary

Term	Meaning
Adapter	Component that maps a provider's response into the normalised `PlatformStats` model
DTO	Data Transfer Object: an explicitly shaped response object
Flyway	Database migration tool that applies versioned SQL scripts
JWT	JSON Web Token used as a stateless bearer credential
Live-source gate	`Platform.hasLiveStatsSource()`; decides if a platform takes part in normal sync
Snapshot	Dated, source-labelled set of statistics for a user and platform
Source label	Provenance tag on data such as `LEETCODE_REAL` or `CODECHEF_THIRD_PARTY`
Testcontainers	Library that runs real services such as MySQL in Docker for tests

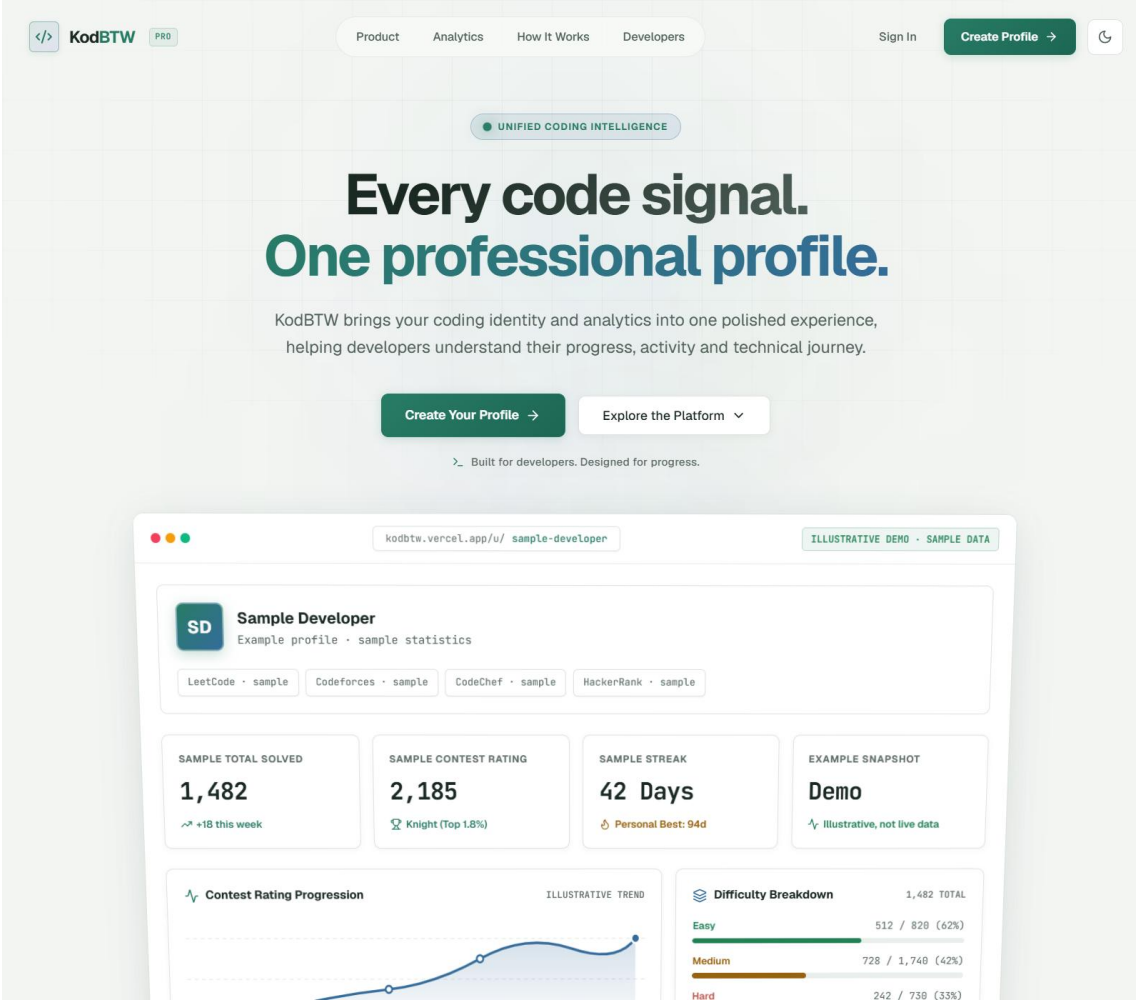
Appendix D: Items Requiring Verification Before Submission

1. Confirm enrolment IDs, department/specialisation (the repository README lists CSE-C / AI&DS / IT as to be confirmed), degree name and session against college records.
2. Confirm the names and designations of the guide and Head of Department, and obtain signatures on the declaration and certificates.
3. Confirm individual contributions with the whole team; none are assigned in this report.
4. Re-run backend and frontend tests on a clean checkout with Docker, including migrations V1–V12, and record the commit ID, date and logs.
5. Capture the remaining login and public-profile views listed as pending in Table 11.2.
6. Confirm the current state of the repository against this report, since later commits may have changed platform status.
7. Confirm provider permissions before describing any source as cleared for production.

Appendix E: Application Snapshots

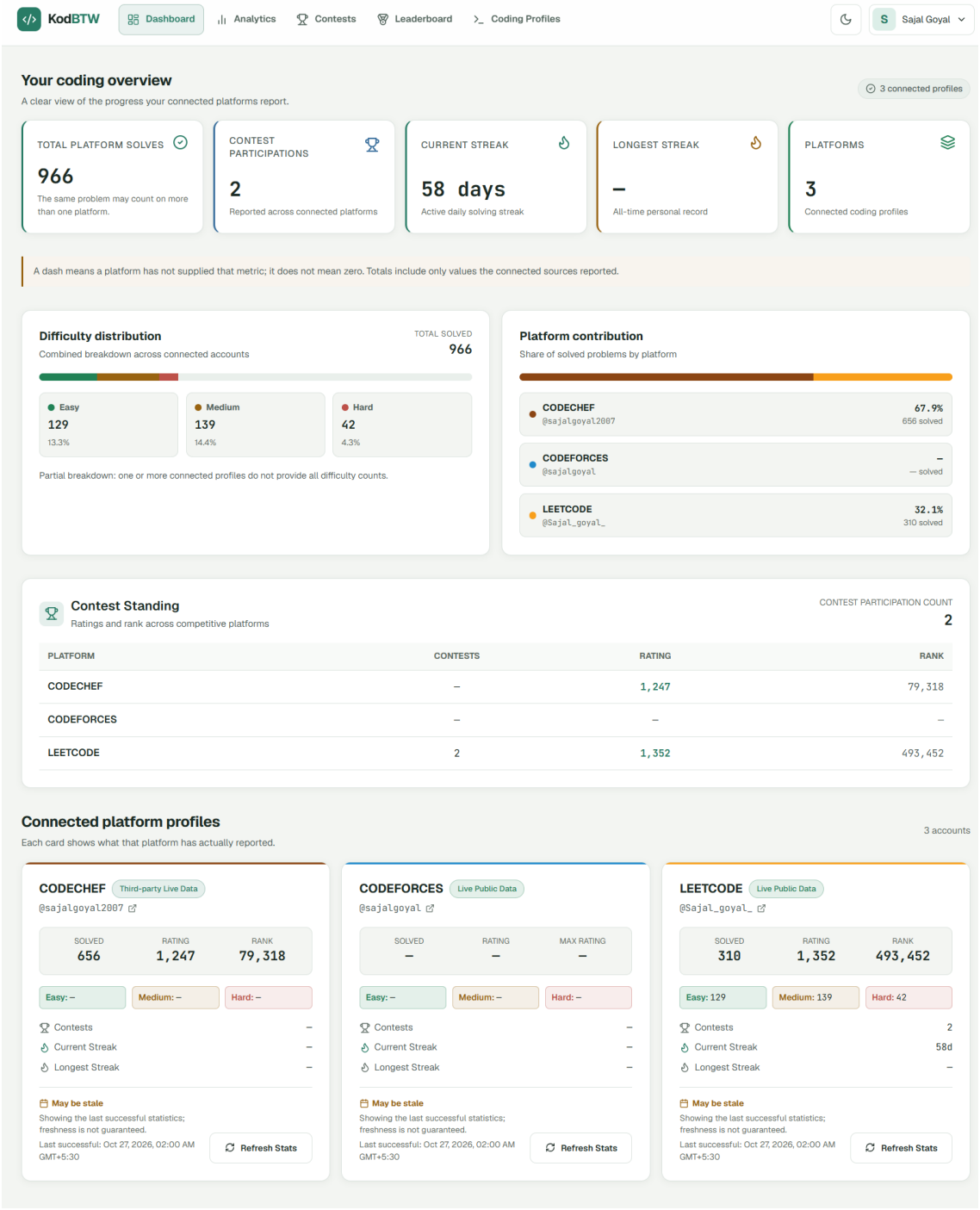
The following supplied screenshots show the KodBTW landing page and application views. Long captures are cropped into consecutive sections for legibility. Values shown are specific to the capture and may change over time.

Figure E.1: KodBTW landing page and product overview.



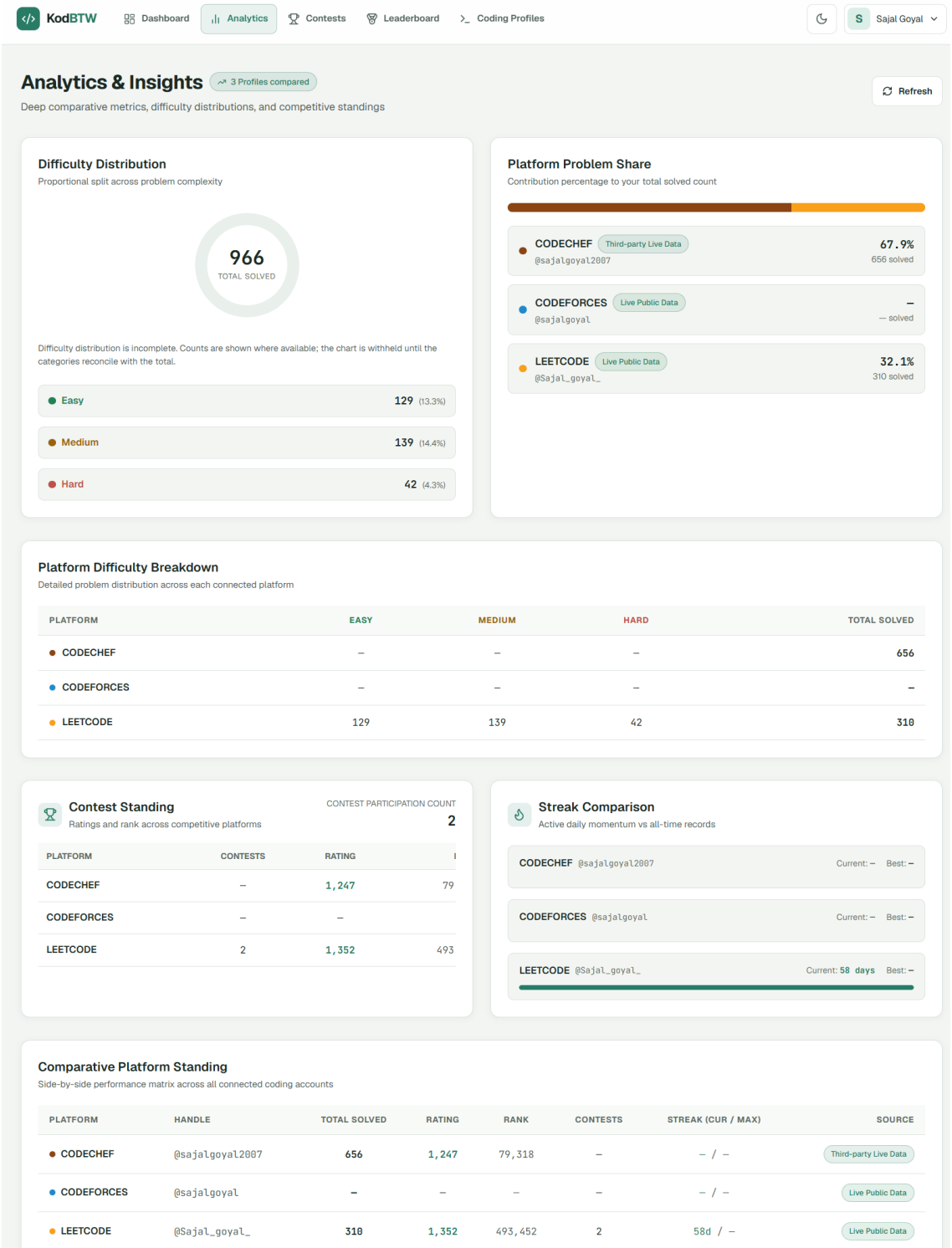
The landing page introduces the unified coding-profile concept and links to the product's main sections. The sample dashboard displayed on the page is labelled illustrative sample data.

Figure E.2: Dashboard overview with connected platform statistics.



The dashboard brings together reported platform totals, difficulty counts, platform share, contest standings and connected-profile cards. A dash indicates that a platform has not supplied that metric.

Figure E.3: Analytics overview and comparative platform metrics.



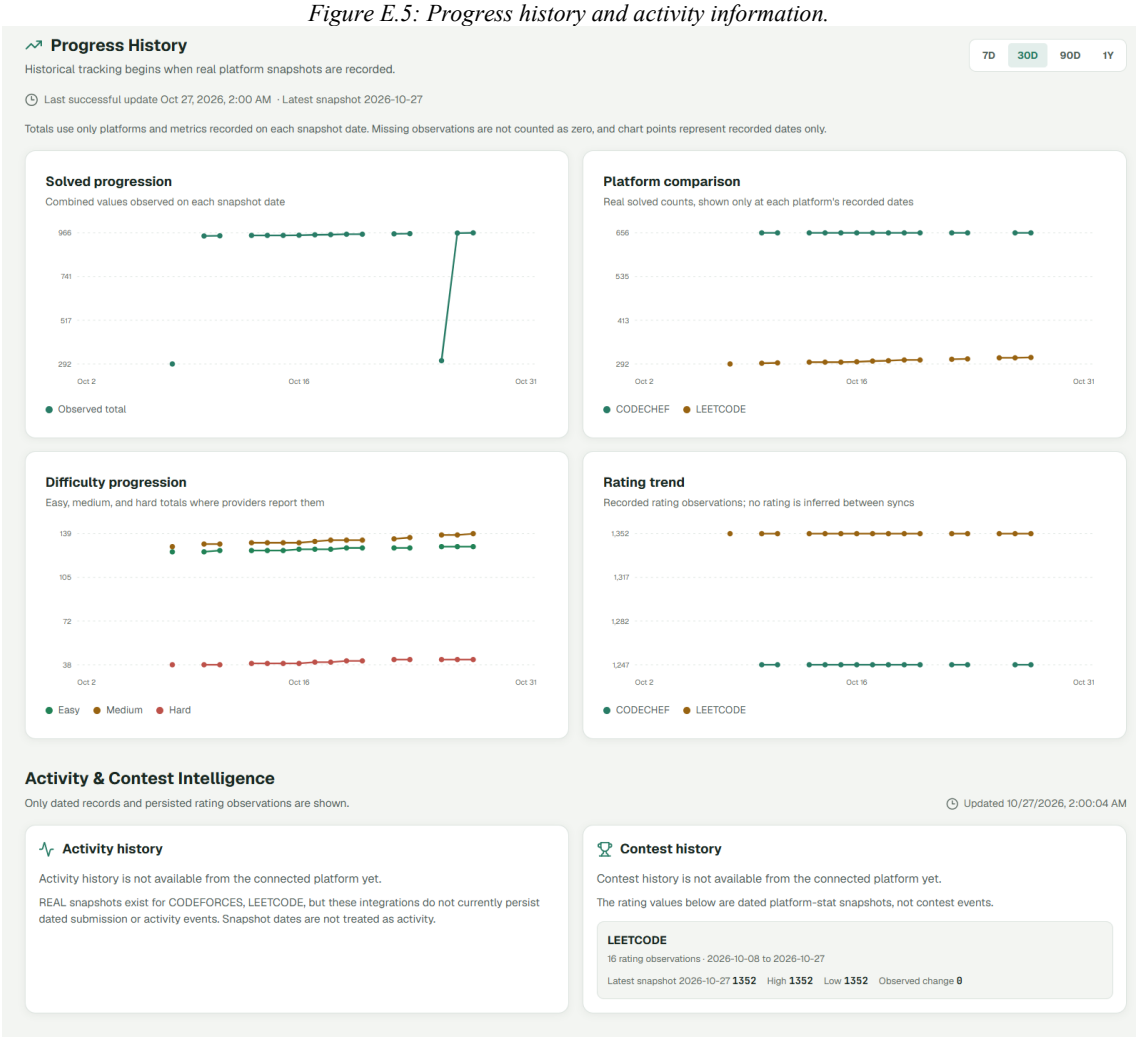
This section compares reported problem counts by difficulty and platform and summarises contest standing and streak data. The view notes that some statistics are incomplete.

Figure E.4: Advanced insights from recorded platform snapshots.



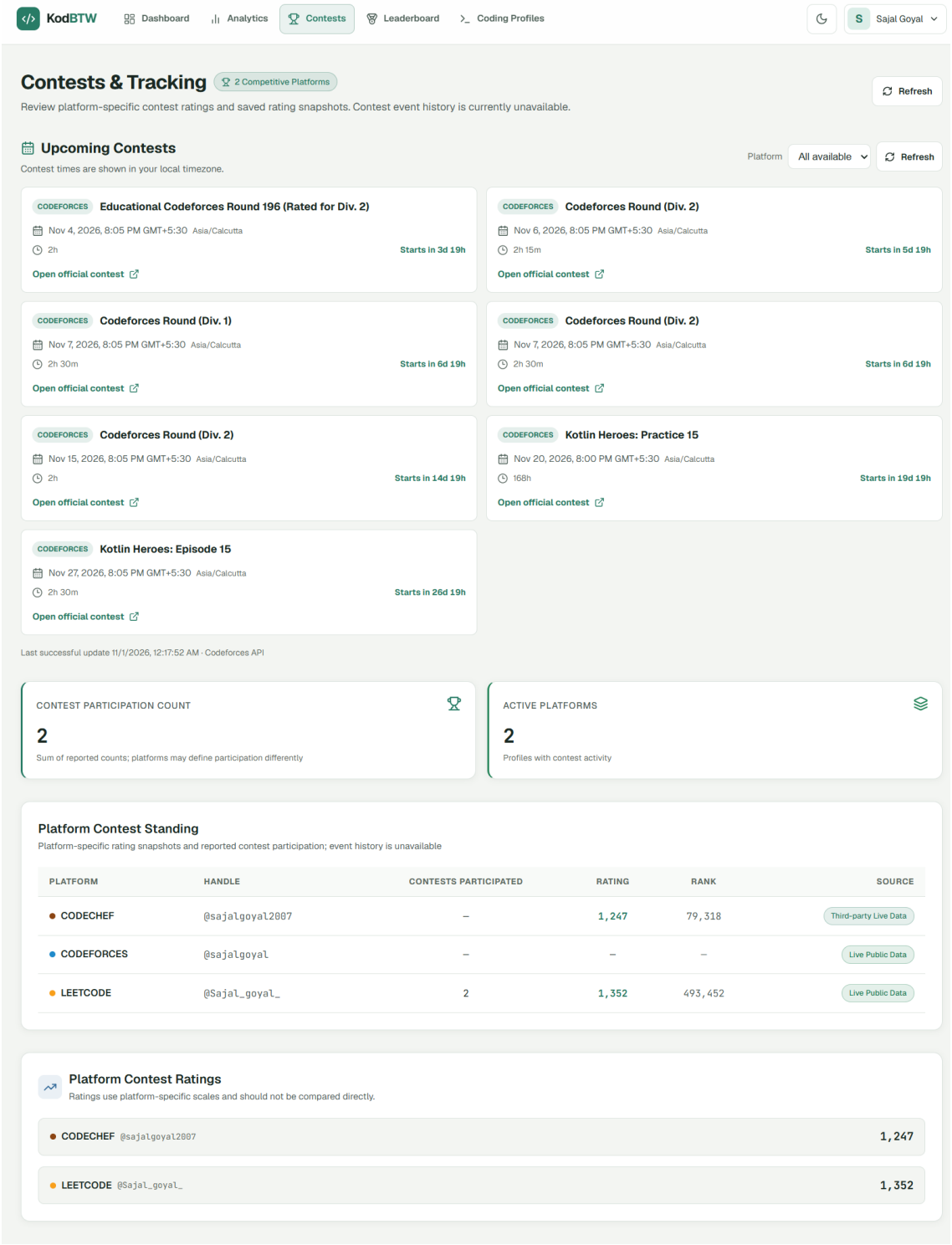
The insights panel summarises dated snapshots and highlights observed changes. It identifies the source and date of records and does not estimate values between synchronisations.

Figure E.5: Progress history and activity information.



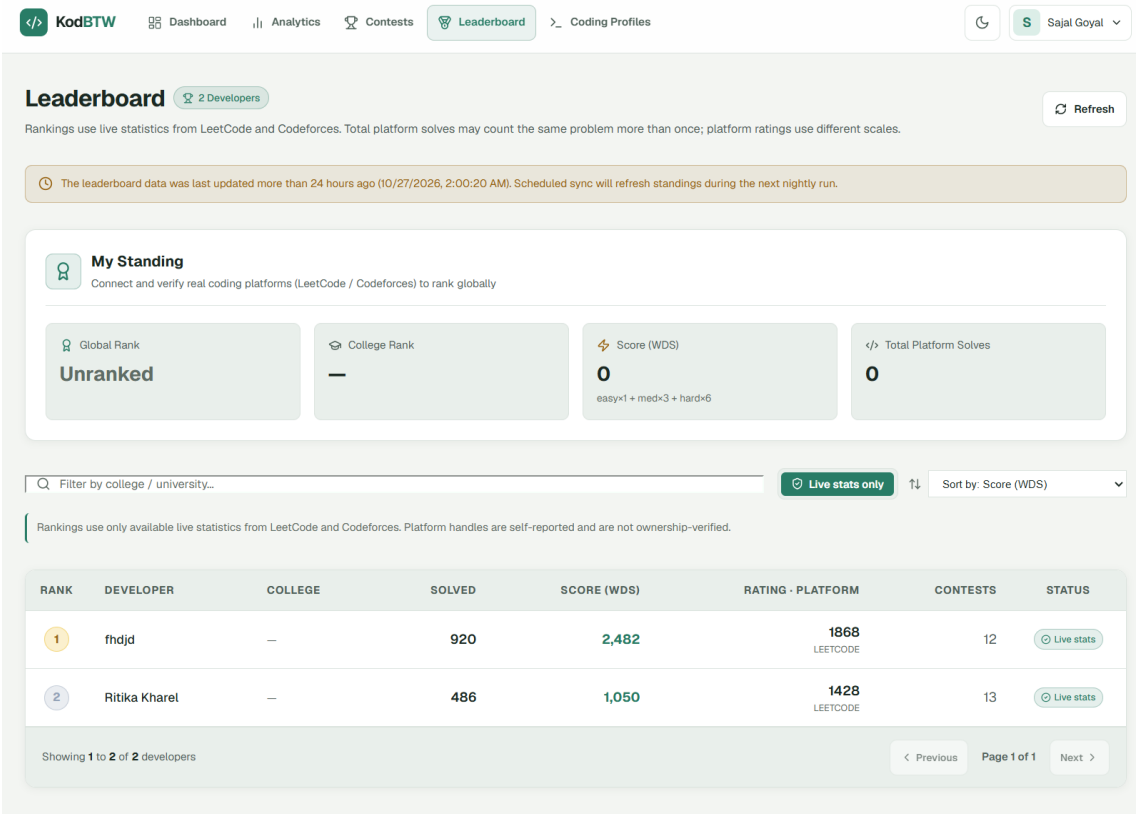
The charts show recorded totals, platform comparisons, difficulty counts and rating observations over time. Activity and contest history are explicitly shown as unavailable where no event records exist.

Figure E.6: Contest calendar and platform contest standings.



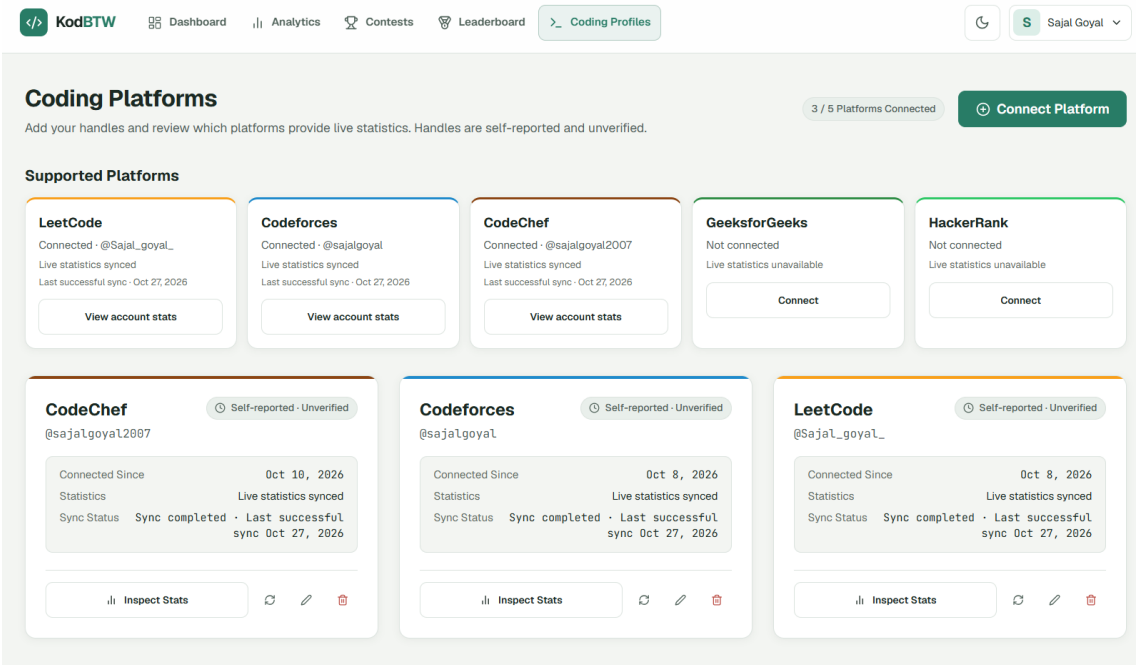
The page lists upcoming contests with local times and shows platform-specific contest ratings and participation data. It also states that contest-event history is unavailable.

Figure E.7: Developer leaderboard and ranking fields.



The leaderboard displays the available live-statistics filter, score, solved count, rating, contest count and source status. A note explains that handles are self-reported and platform rating scales differ.

Figure E.8: Connected coding profiles and synchronization status.



The profiles view lists supported platforms, connected handles, sync status and last successful sync. It identifies handles as self-reported and marks unavailable live statistics for unconnected platforms.